

THE SUPERVISED MACHINE LEARNING WORKFLOW

Using Regression as an Example

Marco R. Steenbergen 

steenbergen@ipz.uzh.ch

University of Zurich

July 19, 2026

R Packages for this Lecture

- **GGally** for bivariate analysis and visualization
- **mlbench** for ML benchmark data sets
- **naniar** for missingness
- **skimr** for univariate summaries
- **tidymodels** for ML
- **tidyverse** for data manipulation

The Generic Supervised Learning Problem

The Learning Goal

i Model

An algorithm learns the model

$$y_i = f(x_i, \theta)$$

Here:

- i is an instance (unit)
- y is the predictive target (outcome variable)—there can be more than one
- f is an unknown function
- x is the feature vector (predictor variables)
- θ contains biases and weights (parameters)

The Prediction Task

Prediction

Once we have learnt the model, we predict:

$$\hat{y}_i = \hat{f}(x_i^*, \hat{\theta})$$

Here,

- $\hat{y}$ is the prediction—a score, class, or probability
- $\hat{f}$ is the learnt function
- x^* is a subset of features (see next slide)
- $\hat{\theta}$ are the learnt biases and weights

The Feature Selection Task

- Supervised ML is not just used for prediction.
- We also want to assess **feature importance**: how important is a feature for the predictive task.
- One step further is **feature selection**:
 - Automatically drop irrelevant features so that $\mathbf{x}^* \subset \mathbf{x}$.
 - Not always done, but can be very useful to guide data collection efforts.

From Model to Prediction

- The learning in ML entails finding $\hat{f}$, $\hat{\theta}$, and $\mathbf{x}^*$.
- The learning space is dictated by the algorithm:
 - Some algorithms are more flexible than others
 - E.g., some allow for automatic interaction detection while others do not
- The learning process can be cast as a statistical **optimization** problem:
 - Within the constraints of the model, we approximate f , θ , and select features to minimize loss.

The Loss Function

- **Loss** = error = any discrepancy between y and $\hat{y}$.
- The **loss function** quantifies the totality of prediction error $L = l(y, \hat{y})$ (the boldface means we collect targets and predictions in vectors).
- Different loss functions exist for different tasks.
- However the general principle is always the same:
 - Within the constraints of the algorithm and the feature pool, we seek to **minimize loss**.
 - This is what we mean by optimization

Performance

Metrics

A performance metric maps the quality of predictions after optimization. While there are many performance metrics ([Japkowicz and Shah 2011](#)), they all have this form:

$$P = p(y, \hat{y})$$

The Inductive Dilemma

Resubstitution Error

- Imagine we do the learning and assess the performance on the exact same data.
 - We call this resubstitution.
- Intuitively, this feels like cheating and it is.
- Efron (**1983**) shows that resubstitution results in optimistic performance estimates.
- The lesson: Do not train and evaluate with the same data!

A Dilemma

- Ideally, one would have two data sets:
 - One for training
 - Another for testing (evaluation)
- Frequently, we only have one data set in our hands, however.
- What to do: Split the data.

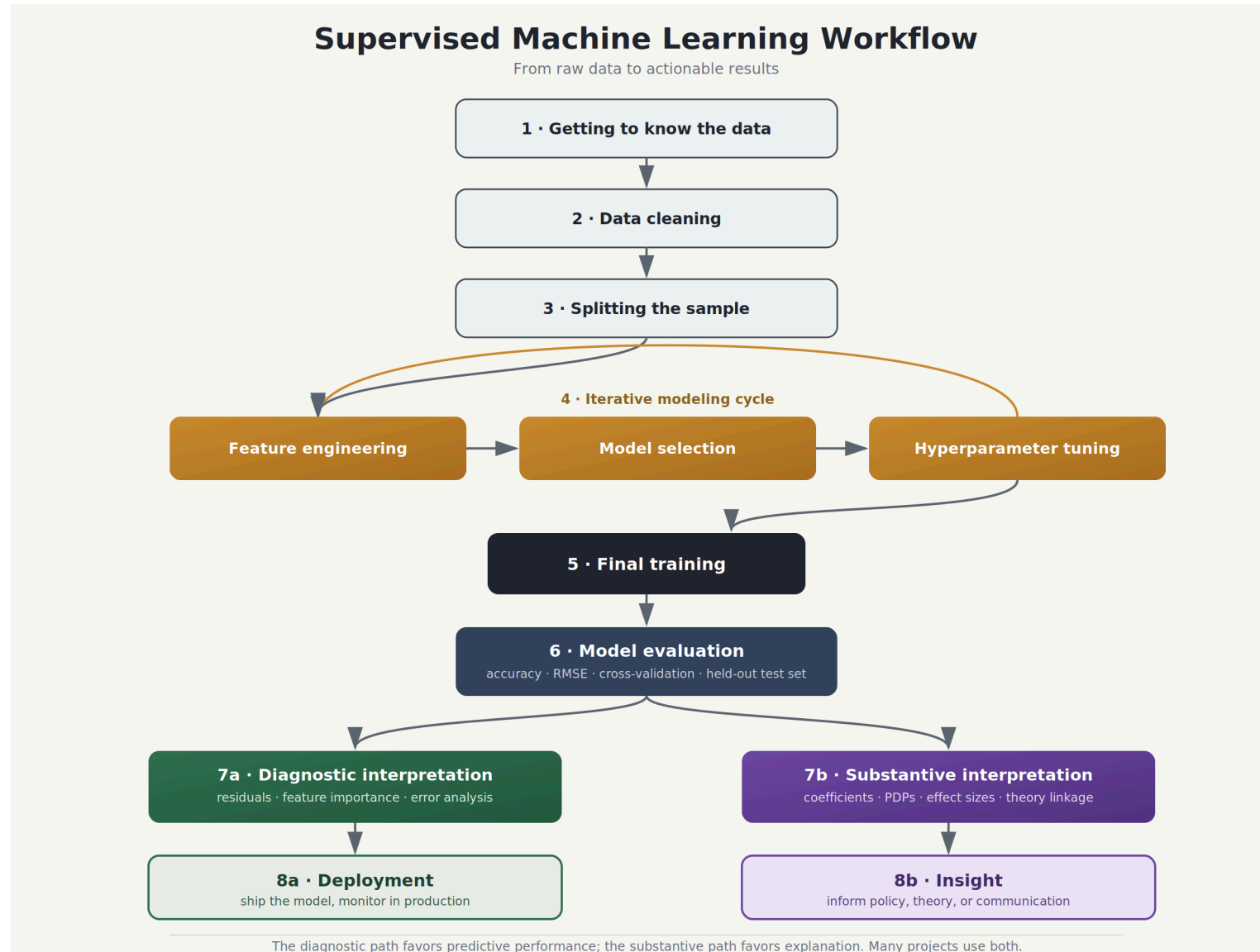
The Split Sample Approach



- If necessary randomize the n instances.
- Set aside a fraction of p instances for training $\Rightarrow$ **training set** has a size of $n_1 = p \cdot n$.
- Set aside a fraction of $1 - p$ instances for evaluation $\Rightarrow$ **test set** has a size of $n_2 = n - n_1$.
- $0.70 \leq p \leq 0.98$, depending on the complexity of the learning task and the amount of data available.

The Machine Learning Workflow

Overview



Getting to Know the Data

- Because ML is data-driven, we should *always* start by taking a look at our data.
- Things you should look at:
 - Missing values.
 - Univariate descriptive statistics and visualizations that could flag lack a lack of variance or outliers.
 - Bivariate descriptions and visualizations that give insight about relationships and possible multicollinearity among features.

Data Cleaning

- Gross errors in the data should be corrected—removal of duplicates and structural errors (e.g., mis-matched strings).
- However, beware of **data leakage**:
 - Information from outside the training dataset is inadvertently used to create or tune a machine learning model.
 - E.g., do not do imputation with the mean across all of the data—this contaminates the future training data with information from the future test set.

Data Splitting

- Instances from the original data are randomly assigned to test and training sets.
- We can do this in a variety of ways:
 - Simple random sampling.
 - Stratified random sampling based on the target or a feature.
- For reproducibility always set a seed.

Feature Engineering

- **Feature engineering** = the process of using domain knowledge to transform raw data into features that expose the underlying problem to a machine learning algorithm.
- This includes:
 - Feature transformation
 - Feature construction
 - Feature extraction (PCA) and pre-selection

Model Selection

- Most algorithms do not produce a single but, in fact, many models.
- This is because those algorithms have **hyper-parameters**: parameters that affect the behavior of the algorithm and cannot be estimated.
 - These parameters are the “knobs” data scientists turn to adjust the behavior of the algorithm.
 - Each constellation of hyper-parameters produces a slightly different model.
- Model selection is the process of determining what the best setting of the hyper-parameters is.

Tuning

- **Tuning** = the process of generating candidate hyper-parameters.
- There are different procedures for generating these candidates:
 1. Grid search
 2. Random search
 3. Sequential model-based optimization (SMBO)
- We use validation to ascertain the “best” hyper-parameter constellation.

Final Training

- Once the best model has been selected, we produce the final fit.
- We freeze the optimal hyper-parameter values.
- We then train the model one more time.

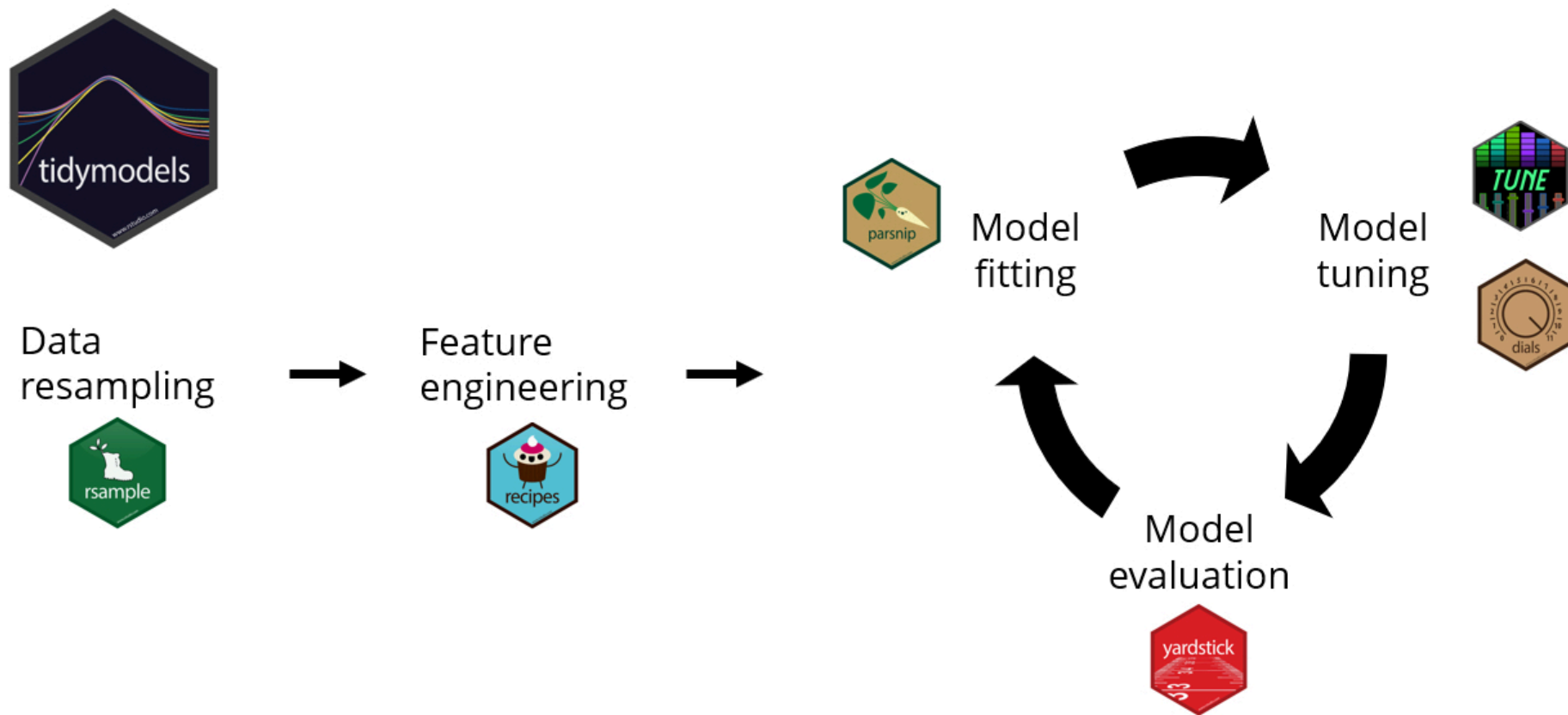
Evaluation

- Only after final training will we go to the test set to assess predictive performance.
- At this stage, we compute performance metrics.
- Those metrics in large part drive whether we continue with the algorithm or go back to the drawing board.

A Fork in the Road

- We can now proceed in two directions:
 1. Use the algorithm to automate a task, for instance, to generate target values for completely new data (deployment).
 2. Use the algorithm for substantive purposes, e.g., generating new insights about a phenomenon.
- In both cases, we may want to engage in both diagnostic and substantive interpretation, but their relative weights change.
- Before deployment it is crucial to assess which predictors influence predictions the most and where the model fails.
- Before claiming new insights, we must scrutinize the substantive effects of predictors using partial dependence plots or other methods.

The Computational Workflow in `tidymodels`



Regression as a Machine

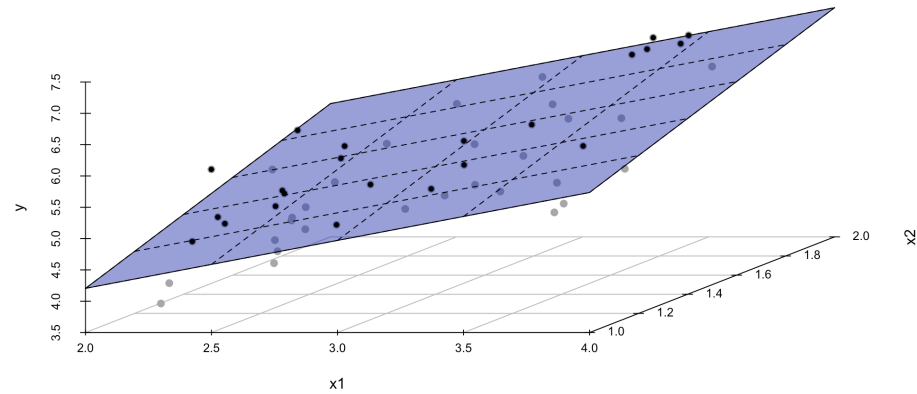
The Regression Model

- We do not typically think of regression as a machine learning algorithm, but it is.
- The target is a numeric score—regression task.
- We have the following model:

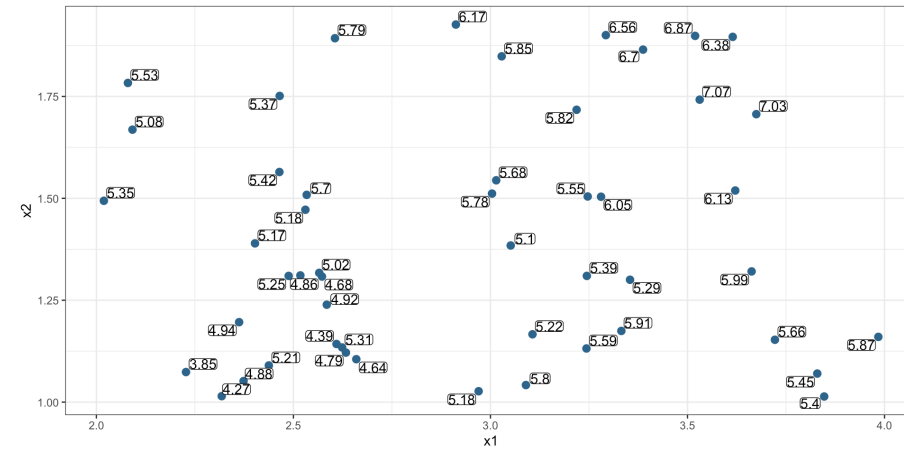
$$y_i = \underbrace{\alpha}_{\text{Bias}} + \sum_{j=1}^P \underbrace{\beta_j}_{\text{Weight}} \cdot x_{ij} + \text{error}$$

Visualization in 2 ways

Hyperplane



Feature Space



Loss Function

- We use a training set to learn $\alpha, \beta_1, \dots, \beta_P$.
- A typical loss function is the L_2 -norm (cf. ordinary least squares):

$$L = \sum_{i=1}^{n_1} \left(y_i - \underbrace{\left\{ \alpha + \sum_j \beta_j \cdot x_{ij} \right\}}_{\hat{y}_i} \right)^2$$

- We minimize the loss by finding the best possible values of the bias and weights in the training set.

Performance

- We apply the trained biases and weights and use that to make predictions in the test set.
- Those predictions are compared to the actual target values in the test set.
- Common performance metrics are:



$$\text{RMSE: } P = \sqrt{\sum_{i=1}^{n_2} (y_i - \hat{y}_i)^2}.$$



$$\text{R-squared: } R^2 = \text{cor}_{y_i, \hat{y}_i \in \text{test set}}^2.$$

Building Our First Workflow

The Political Economy of Housing Values

- Harrison and Rubinfeld (**1978**) present data on median housing values in census tracts in the Boston area.
- They are particularly interested in the effect of pollution on housing values.
- We also add property taxes to our analysis.
- The data can be found in the **mlbench** package.

Data Screening

Data

Missing

Univariate

Bivariate 1

Bivariate 2

```
1 library(mlbench)
2 library(tidyverse)
3 data("BostonHousing2")
4 df <- BostonHousing2 |>
5   select(cmedv, nox, tax, tract)
```

Data Screening

Data

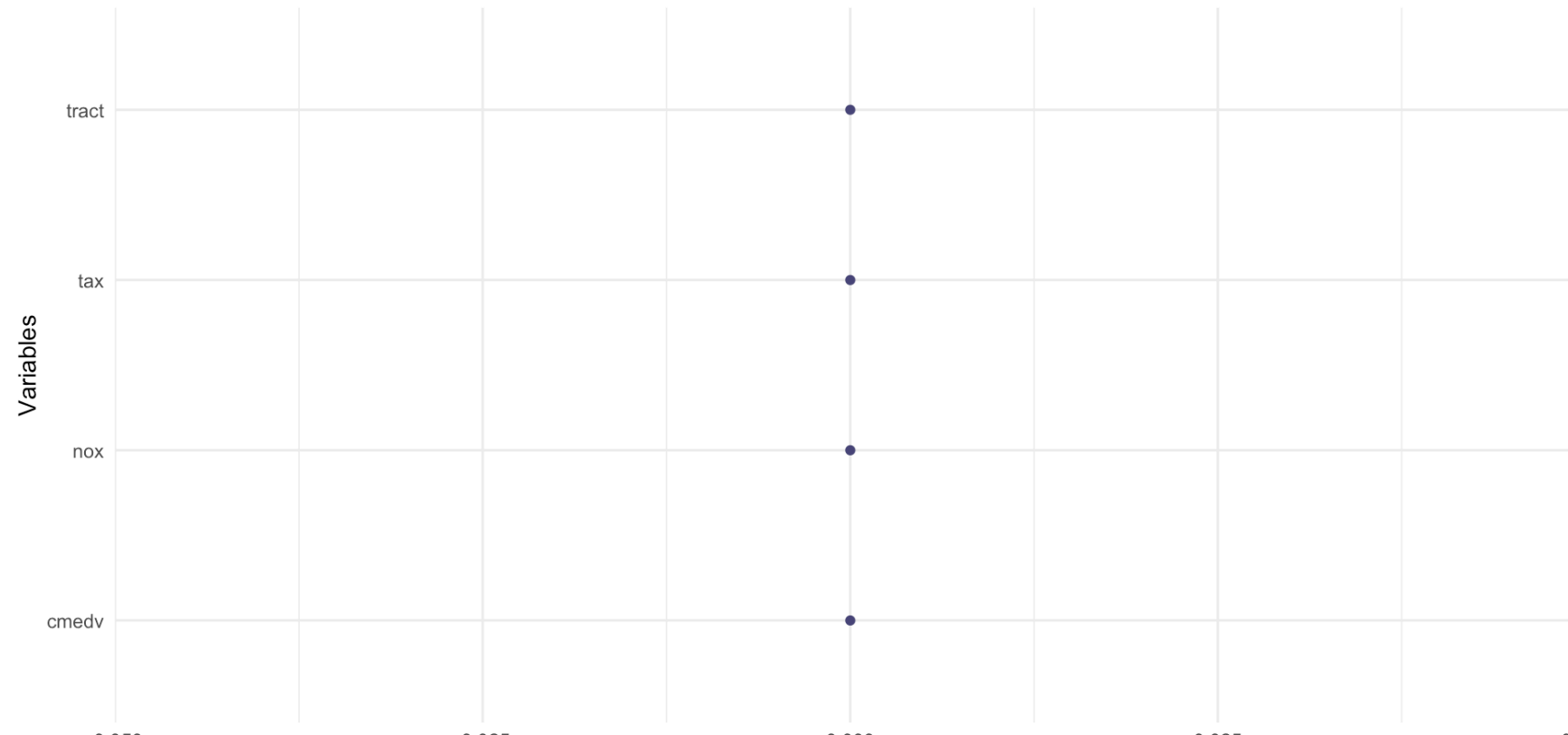
Missing

Univariate

Bivariate 1

Bivariate 2

```
1 library(naniar)
2 gg_miss_var(df, show_pct = FALSE) +
3   labs(y = "# Missing") +
4   theme_minimal()
```



Data Screening

Data

Missing

Univariate

Bivariate 1

Bivariate 2

```
1 library(skimr)
2 skimmed <- skim(df[,1:3])
3 yank(skimmed, "numeric")
```

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p1
cmedv	0	1	22.53	9.18	5.00	17.02	21.20	25.00	50.
nox	0	1	0.55	0.12	0.38	0.45	0.54	0.62	0.
tax	0	1	408.24	168.54	187.00	279.00	330.00	666.00	711.

Data Screening

Data

Missing

Univariate

Bivariate 1

Bivariate 2

```
1 library(GGally)
2 df_small <- df[,1:3]
3 p <- ggpairs(df_small,
4             lower = list(continuous = "smooth_loess"),
5             upper = list(continuous = wrap(ggally_cor, displayGrid = FALSE)))
6 p + theme_minimal()
```

Data Screening

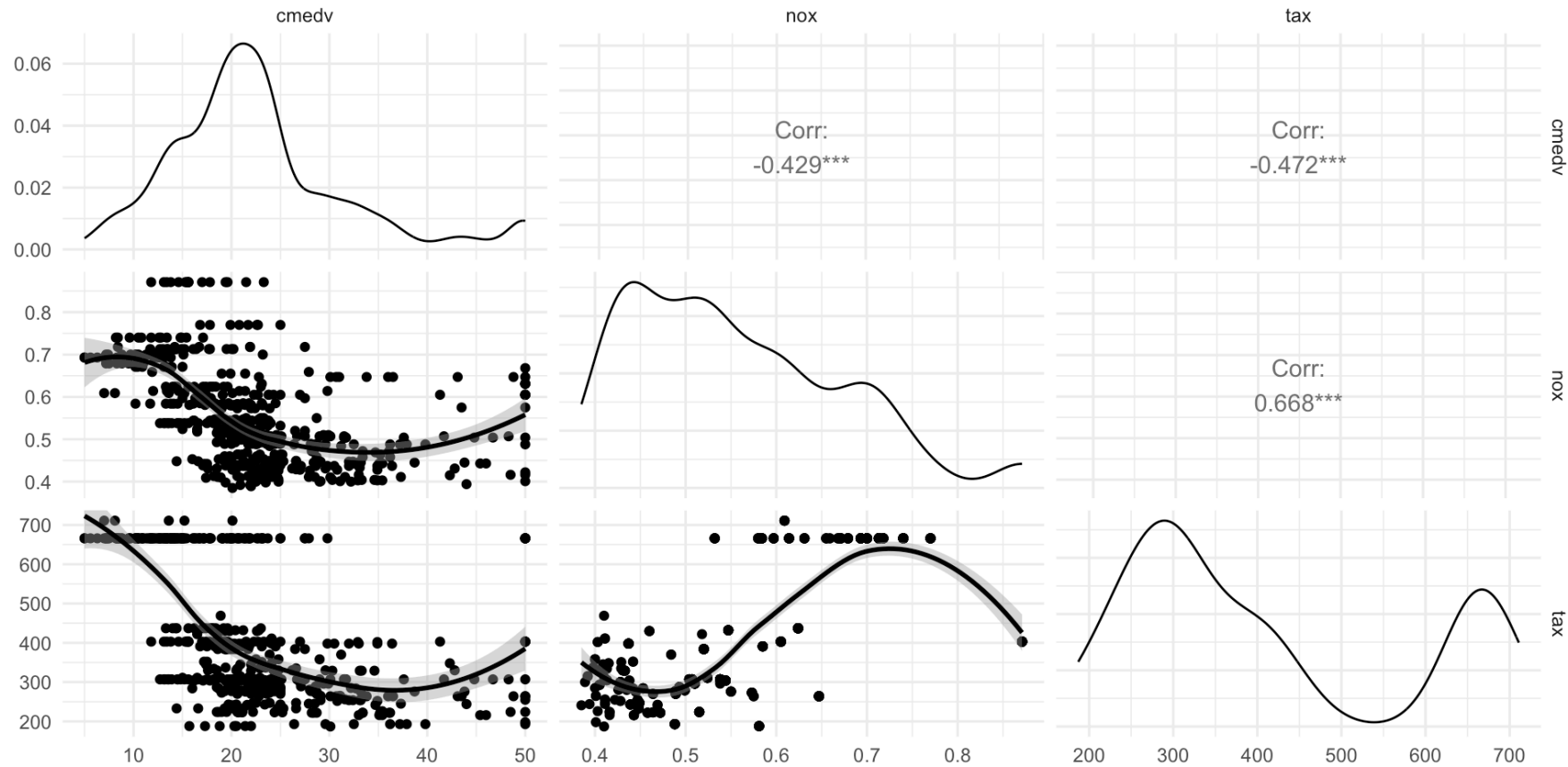
Data

Missing

Univariate

Bivariate 1

Bivariate 2



Conclusion from Screening

- Skewness $\Rightarrow$ Box-Cox transformation (**Box and Cox 1964**) or logarithm.
 - $\rightarrow$ High risk of data leakage when estimation involved (Box-Cox).
 - $\rightarrow$ Must be done after the split using the **prep()** and **bake()** logic of **tidymodels**. However, target is the exception.
- Bi-modality due to cases with high property tax rates $\Rightarrow$ add dummy:
 - $\rightarrow$ Suburbs with high accessibility to radial highways—structural property.
 - $\rightarrow$ This can be done before the split because no estimation needs to be done.

Initial Split

Data

Split

Apply

```
1 df_new <- df |>
2   mutate(target = log10(cmedv),
3           high_tax = ifelse(tax > 600, 1, 0)) |>
4   select(tract, target, nox, tax, high_tax)
```

Initial Split

Data

Split

Apply

```
1 library(tidymodels)
2 set.seed(1292)
3 housing_split <- initial_split(df_new, prop = 0.75)
4 housing_split
```

<Training/Testing/Total>
<379/127/506>

Initial Split

Data

Split

Apply

```
1 train_df <- training(housing_split)
2 test_df <- testing(housing_split)
```

Feature Engineering

Goal

Syntax

Result

On the training data, we do the following:

- Change the role of “tract” to that of an identifier.
- Perform a Box-Cox transformation on “nox” and “tax”

Feature Engineering

Goal

Syntax

Result

```
1 housing_recipe <- recipe(target ~ ., data = train_df) |>
2   update_role(tract, new_role = "id") |>
3   step_BoxCox(nox, tax)
```

Feature Engineering

Goal

Syntax

Result

— Recipe

— Inputs

Number of variables by role

outcome: 1

predictor: 3

id: 1

— Operations

- Box-Cox transformation on: nox tax

Model and Workflow

```
1 housing_model <- linear_reg() |>
2   set_mode("regression") |>
3   set_engine("lm")
4 housing_wf <- workflow() |>
5   add_model(housing_model) %>%
6   add_recipe(housing_recipe)
```

Training

Comment	Syntax	Results 1	Results 2
• There are no hyper-parameters. • We do not need to tune and can go straight to the final fit.			

Training

Comment

Syntax

Results 1

Results 2

```
1 housing_fit <- fit(housing_wf, train_df)
```

Training

Comment

Syntax

Results 1

Results 2

```
1 tidy(housing_fit)
```

```
# A tibble: 4 × 5
```

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>
1	(Intercept)	5.35	1.54	3.47	0.000588
2	nox	-0.166	0.0333	-4.97	0.00000101
3	tax	-2.29	0.851	-2.69	0.00749
4	high_tax	-0.0612	0.0317	-1.93	0.0541

Training

Comment

Syntax

Results 1

Results 2

```
1 glance(housing_fit)
```

```
# A tibble: 1 × 12
```

	r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.351	0.346	0.144	67.7	5.47e-35	3	199.	-388.	-368.

```
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Performance

Prediction

Metrics

```
1 housing_fit |>  
2   predict(test_df) |>  
3   bind_cols(test_df)
```

A tibble: 127 × 6

	.pred	tract	target	nox	tax	high_tax
*	<dbl>	<int>	<dbl>	<dbl>	<int>	<dbl>
1	1.42	2021	1.33	0.469	242	0
2	1.44	2032	1.56	0.458	222	0
3	1.44	2033	1.46	0.458	222	0
4	1.36	2042	1.34	0.524	311	0
5	1.36	2045	1.18	0.524	311	0
6	1.36	2046	1.28	0.524	311	0
7	1.35	2054	1.36	0.538	307	0
8	1.35	2063	1.14	0.538	307	0
9	1.35	2068	1.10	0.538	307	0
10	1.35	2070	1.12	0.538	307	0

i 117 more rows

Performance

Prediction

Metrics

```
1 housing_fit |>  
2   predict(test_df) |>  
3   bind_cols(test_df) |>  
4   metrics(truth = target, estimate = .pred)
```

A tibble: 3 × 3

	.metric	.estimator	.estimate
	<chr>	<chr>	<dbl>
1	rmse	standard	0.140
2	rsq	standard	0.360
3	mae	standard	0.106

Exercise

Your Turn

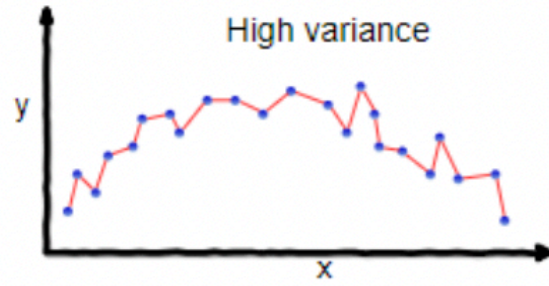
Add two additional features to the model and repeat the workflow.

The Risks of Machine Learning

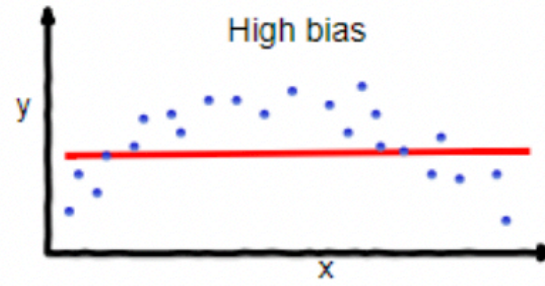
Machine Learning Errors

Error Source	Description	Cause(s)
Irreducible error	Error that cannot be eliminated	(1) missing features; (2) measurement error; and (3) random shocks
Bias error	Systematic prediction error	Under-fitting
Variance error	Lack of generalizability of the model	Over-fitting—too much flexibility

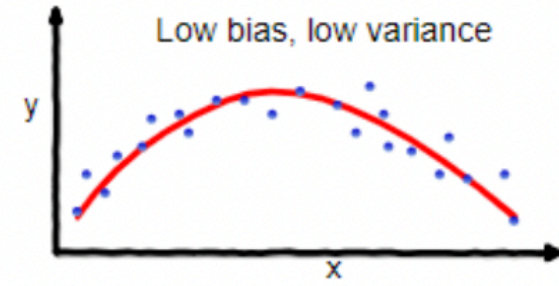
Illustrating Bias and Variance Error



overfitting



underfitting



Good balance

Source: [Seema Singh](#)

The Bias-Variance Trade-off

- Flexibility $\uparrow$, bias error $\downarrow$ but variance error $\uparrow$.
- Flexibility $\downarrow$, variance error $\downarrow$ but bias error $\uparrow$.

We Need to Consider Both Bias and Variance

- **Mean squared error:**

$$\text{MSE} = \text{Bias}^2 + \text{Variance}$$

- We want to minimize MSE.
- Sometimes that means introducing a bit of bias in order to reduce the variance $\Rightarrow$ regularization.

References

- Box, G. E. P., and D. R. Cox. 1964. "An Analysis of Transformations." *Journal of the Royal Statistical Society. Series B (Methodological)* 26 (2): 211–52. <http://www.jstor.org/stable/2984418>.
- Efron, Bradley. 1983. "Estimating the Error Rate of a Prediction Rule: Improvement on Cross-Validation." *Journal of the American Statistical Association* 78 (382): 316–31. <https://doi.org/10.1080/01621459.1983.10477973>.
- Harrison, David, and Daniel L Rubinfeld. 1978. "Hedonic Housing Prices and the Demand for Clean Air." *Journal of Environmental Economics and Management* 5 (1): 81–102. [https://doi.org/10.1016/0095-0696\(78\)90006-2](https://doi.org/10.1016/0095-0696(78)90006-2).
- Japkowicz, Nathalie, and Mohak Shah. 2011. *Evaluating Learning Algorithms: A Classification Perspective*. Cambridge University Press. <https://www.cambridge.org/core/books/evaluating-learning-algorithms/3CB22D16AB609D1770C24CA2CB5A11BF>.